SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Testing Strategies for Reliable Embedding-Driven ML Systems: A Case Study on Biocentral Services

Author:	Deniz Özturk
Examiner:	Sebastian Franz
Supervisor:	Advisor
Submission Date:	Submission date

Deniz Özturk. *Testing Strategies for Reliable Embedding-Driven ML Systems: A Case Study on Biocentral Services*. bachelor's thesis, Technische Universität München, Munich, Germany, 2026.

Supervised by Sebastian Franz; submitted on March 30, 2026 to the School of Computation, Information and Technology of the Technische Universität München.

© 2026 Deniz Özturk

Chair of Bioinformatics, Technische Universität München, Boltzmannstraße 3, 85748 Garching bei München, Germany, `https://www.in.tum.de/bio/`.

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license, visit `http://creativecommons.org/licenses/by/4.0/` or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Transparency in the Use of AI Tools

In preparing this thesis, I utilized Grammarly to correct grammatical errors, improve sentence structure, and enhance overall writing style. I used ChatGPT to generate initial drafts, expand on ideas, providing valuable suggestions and examples. I have carefully checked all texts created with these tools to ensure that they are correct and make sense.

Abstract

Protein language models (PLMs) such as ESM2 and ProtT5 are increasingly integrated into production bioinformatics platforms, yet systematic approaches to testing these embedding-driven systems remain scarce. The absence of ground-truth oracles, GPU-induced non-determinism, and high computational cost make traditional software testing techniques insufficient.

This thesis presents a comprehensive, multi-layer testing strategy for PLM-based services, developed and evaluated as a case study on the biocentral server—an open-source platform for embedding-based protein analysis. The strategy comprises five layers: unit tests with mocked dependencies, integration tests against the live server, property-based oracle tests verifying mathematical invariants, performance benchmarks, and metamorphic experiments validating PLM embedding behavior at scale.

Key contributions include (1) a deterministic, GPU-free testing infrastructure (*FixedEmbedder*) that enables the full integration test suite to run in CI without GPU resources, (2) eight metamorphic experiments—including progressive X-masking, reversed sequence sensitivity, and amino acid mutation sensitivity—validated on 1,750 real protein sequences from UniRef50, (3) a reusable oracle testing framework with statistical utilities for robust invariant checking, and (4) mutation testing evaluation of the test suite's fault-detection capability.

The evaluation quantifies the trade-off between CI execution time and test coverage across all layers, providing concrete recommendations for allocating testing budgets in embedding-driven ML services. Results demonstrate that integration tests with the FixedEmbedder offer the strongest cost-effectiveness ratio, while metamorphic experiments provide unique behavioral insights into PLM properties that no other test layer captures.

Contents

Transparency in the Use of AI Tools **3**

Abstract **4**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Question 3
1.4 Contributions 3
1.5 Thesis Outline 4

2 Background **5**
2.1 Protein Language Models 5
2.2 The Biocentral Platform 6
2.3 Software Testing Fundamentals 7
2.4 Challenges of Testing ML Systems 8
2.5 Metamorphic Testing 9
2.6 Property-Based and Oracle Testing 9

3 Related Work **11**
3.1 Testing Machine Learning Systems 11
3.2 Metamorphic Testing for Deep Learning 11
3.3 Testing Scientific and Bioinformatics Software 12
3.4 Property-Based Testing for Data Pipelines 12
3.5 Research Gap 13

4 Approach **14**
4.1 Overall Test Architecture 14
4.2 Test Infrastructure 14
4.2.1 Canonical Test Dataset 14
4.2.2 FixedEmbedder 15
4.2.3 Integration Test Infrastructure 15
4.3 Unit Tests 16

4.4 Integration Tests . . . 16
4.5 Property-Based Oracle Tests . . . 17
4.6 Metamorphic Experiments . . . 18
4.6.1 Idempotency . . . 18
4.6.2 Batch Invariance . . . 18
4.6.3 Progressive X-Masking . . . 19
4.6.4 Reversed Sequence . . . 19
4.6.5 Amino Acid Mutation Sensitivity . . . 19
4.6.6 Random Baseline . . . 20
4.6.7 Projection Determinism . . . 20
4.6.8 Training Determinism . . . 20
4.7 Performance Tests . . . 20
4.8 Mutation Testing . . . 21

5 Evaluation **22**
5.1 Experimental Setup . . . 22
5.2 Test Coverage and Mutation Testing Results . . . 22
5.3 CI Execution Time Analysis . . . 23
5.4 Metamorphic Experiment Results . . . 23
5.4.1 X-Masking Results . . . 23
5.4.2 Reversed Sequence Results . . . 24
5.4.3 AA Mutation Sensitivity Results . . . 24
5.5 Oracle Test Findings . . . 25
5.6 Performance Results . . . 25
5.7 Discussion of Results . . . 25

6 Discussion **27**
6.1 Trade-Off Summary . . . 27
6.2 Generalizability . . . 27
6.3 Limitations . . . 28
6.4 Threats to Validity . . . 28

7 Conclusion **30**
7.1 Summary of Contributions . . . 30
7.2 Key Findings . . . 30
7.3 Future Work . . . 31

Abbreviations **32**

List of Figures **33**

List of Tables **34**

Bibliography **35**

1 Introduction

1.1 Motivation

Protein Language Model (PLM)-based services have become a central component in computational biology research. Models such as ESM2 and ProtT5 encode protein sequences into high-dimensional embedding vectors that capture structural and functional properties learned from large-scale pre-training on protein databases [Eln+21]. These embeddings enable a broad range of downstream analyses, including secondary structure prediction, subcellular localization, and variant effect estimation, without requiring explicit feature engineering. As a result, embedding-driven systems are increasingly deployed as production services that expose PLM capabilities through Application Programming Interface (API) endpoints.

Biocentral is one such system: an open-source, production-grade platform for embedding-based protein analysis [Fra+25]. Its server component provides nine API modules spanning embedding computation, prediction serving, dimensionality reduction, custom model training, and Bayesian optimization. The platform relies on Graphics Processing Unit (GPU)-accelerated inference via NVIDIA Triton Inference Server, asynchronous task pipelines backed by Redis, and a PostgreSQL database for embedding caching. This architecture enables scalable protein analysis but introduces substantial complexity in the software stack that must be validated through systematic testing.

Traditional software testing relies on two fundamental assumptions: that program outputs are deterministic for a given input, and that expected outputs are known in advance. Neither assumption holds for Machine Learning (ML) embedding systems. PLM embeddings lack a ground-truth oracle; there is no reference output against which a given embedding vector can be verified as correct. GPU-based floating-point arithmetic introduces non-determinism across hardware configurations and driver versions, meaning that identical inputs may produce numerically different outputs across runs [Cha+23]. Furthermore, the computational cost of loading and executing large transformer models renders exhaustive test execution impractical within typical Continuous Integration (CI) time budgets. These characteristics—the absence of an oracle, hardware-induced non-determinism, and prohibitive compute requirements—distinguish ML embedding services from conventional software systems and demand

testing approaches specifically adapted to these constraints.

Despite the growing deployment of PLM-based services, the problem of testing such systems remains under-explored relative to the attention devoted to model development and training. Research on CI practices in ML projects indicates that testing is frequently deprioritized in favor of model performance optimization [Ber+25; Rzi+25]. This imbalance is problematic: deployment-grade ML services must satisfy not only model-level accuracy requirements but also system-level correctness guarantees covering API contracts, data pipeline integrity, caching consistency, and inference reproducibility.

1.2 Problem Statement

The Biocentral server integrates nine API modules, GPU-accelerated inference through Triton, asynchronous task pipelines with worker processes, and database-backed embedding caching into a single deployable system. Each of these components introduces distinct failure modes: API endpoints may violate their contracts, the inference server may produce inconsistent results across model versions, task pipelines may lose or corrupt intermediate state, and the caching layer may serve stale or incorrectly keyed embeddings. Ensuring correctness across this heterogeneous stack requires a test strategy that addresses each layer individually while also validating the interactions between layers.

Conventional unit and integration tests are necessary but insufficient for this setting. Unit tests verify isolated functions but cannot detect faults that arise from component interactions or from the statistical nature of ML outputs. Integration tests validate system-level workflows but depend on infrastructure availability and do not address the oracle problem inherent in embedding computation. Property-based and metamorphic testing techniques offer a principled approach to validating ML behavior in the absence of ground truth—for example, by asserting that semantically similar inputs produce similar embeddings, or that known sequence transformations induce predictable changes in embedding space. However, these techniques require executing real PLM inference, which is computationally expensive and typically demands GPU hardware.

This creates a fundamental tension between test thoroughness and CI execution time. A comprehensive test suite that includes property-based and metamorphic tests over large protein datasets may require hours of GPU computation, rendering it impractical for execution on every commit. Conversely, a fast test suite restricted to unit and lightweight integration tests may fail to detect faults specific to ML inference behavior. The central problem addressed in this thesis is therefore the design and evaluation of a multi-layer test strategy for embedding-driven ML services that balances fault-detection

capability against the practical constraints of CI pipelines.

1.3 Research Question

The preceding observations motivate the following research question:

> Research Question (RQ): *What is the trade-off between CI execution time and fault-detection capability when applying property-based and metamorphic testing to embedding-driven ML services?*

This question is decomposed into two sub-questions. First, which test layers—unit, integration, property-based, or metamorphic—provide the highest fault-detection value per unit of CI time? Second, can GPU-free deterministic embedders replace real PLM models in CI pipelines without reducing the structural coverage achieved by the test suite?

1.4 Contributions

This thesis makes the following contributions:

1. A multi-layer test pyramid tailored to embedding-driven ML services, organized into five layers: unit tests, integration tests, property-based and oracle tests, performance benchmarks, and metamorphic experiments.
2. `FixedEmbedder`, a SHA-256-based deterministic embedder that produces reproducible embedding vectors from protein sequences without requiring GPU hardware or neural network inference. This component enables the full CI pipeline to execute on commodity hardware.
3. Eight metamorphic experiments that validate PLM embedding behavior under controlled sequence transformations, including unknown-token masking, sequence reversal, and single-residue mutation sensitivity, evaluated at UniRef50 scale.
4. An oracle test pattern supported by statistical utilities for confidence interval estimation, effect size computation, and significance threshold evaluation, enabling quantitative assertions on ML outputs.
5. A mutation testing evaluation using `mutmut` that quantifies the fault-detection capability of the unit test suite.

6. A quantitative analysis of the trade-off between CI execution time and fault-detection coverage across the test layers, providing empirical guidance for test portfolio selection.

1.5 Thesis Outline

The remainder of this thesis is organized as follows.

Chapter 2 introduces the foundational concepts underlying this work, including PLM architectures and their embedding representations, the Biocentral platform and its server-side components, and software testing fundamentals from the test pyramid to property-based and metamorphic testing.

Chapter 3 surveys related work on testing ML systems, covering metamorphic testing for deep learning, testing practices in scientific software, and property-based testing approaches for data pipelines.

Chapter 4 presents the test architecture, infrastructure design, and experimental methodology. This includes the five-layer test pyramid, the `FixedEmbedder` design, the canonical test dataset, integration test infrastructure, and the metamorphic experiment specifications.

Chapter 5 reports the evaluation results, including code coverage and mutation testing metrics, CI execution time measurements per test layer, and the quantitative trade-off analysis between testing thoroughness and pipeline speed.

Chapter 6 interprets the findings, assesses generalizability to other ML service architectures, acknowledges limitations of the approach, and identifies threats to internal, external, and construct validity.

Chapter 7 summarizes the contributions and outlines directions for future work.

2 Background

This chapter introduces the foundational concepts that underlie the testing strategy developed in this thesis. It covers protein language models and their embedding representations, the Biocentral platform architecture, classical software testing principles, the specific challenges of testing ML systems, and two testing paradigms—metamorphic testing and property-based oracle testing—that address the oracle problem inherent in embedding-driven services.

2.1 Protein Language Models

Proteins are linear chains of amino acids, drawn from an alphabet of 20 standard residues. Additional special tokens represent non-standard or ambiguous residues: X denotes an unknown amino acid, while B, Z, U, and O encode asparagine-or-aspartate ambiguity, glutamine-or-glutamate ambiguity, selenocysteine, and pyrrolysine, respectively. Each protein's biological function is determined by its amino acid sequence, which folds into a three-dimensional structure.

The protein language model paradigm treats amino acid sequences analogously to sentences in natural language. Transformer architectures, originally developed for machine translation, have been adapted to learn representations of protein sequences through self-supervised pre-training on large protein databases. These models employ self-attention mechanisms that capture dependencies between residues at arbitrary sequence positions, combined with positional encodings that preserve information about residue order.

The dominant pre-training objective is masked language modeling: a fraction of residues in each sequence is replaced with a mask token, and the model is trained to predict the original residues from context. Pre-training is performed on large-scale databases such as UniRef [Suz+15] and BFD. Key models include ESM2 from Meta AI, which spans model sizes from 8 million to 15 billion parameters [Lin+23], and ProtTrans, a family of models including ProtT5 that was trained using high-performance computing infrastructure [Eln+21].

These models produce two types of embedding representations. Per-residue embeddings yield a matrix of dimensions $L \times d$, where L is the sequence length and d is the hidden dimension of the model. Pooled embeddings reduce this matrix to a

single vector of dimension d by computing the mean across residue positions, producing a fixed-size representation for each protein regardless of sequence length. These embeddings enable a broad range of downstream tasks without task-specific retraining, including secondary structure prediction, subcellular localization, binding site prediction, disorder prediction, conservation scoring, and variant effect estimation.

For comparison and baseline purposes, simpler encoding schemes exist. One-hot encoding represents each amino acid as a binary vector of length 20, where only the position corresponding to the amino acid is set to one. The BLOSUM62 substitution matrix provides an alternative representation based on observed amino acid substitution frequencies in aligned protein families.

2.2 The Biocentral Platform

Biocentral is an open-source platform for embedding-based protein analysis [Fra+25]. It follows a client-server architecture: a Flutter-based desktop client provides the graphical user interface, while a FastAPI backend handles computation, data management, and ML inference.

The server component integrates several infrastructure services orchestrated via Docker Compose. The core web server is built on FastAPI and exposes a Representational State Transfer (REST) API. Computationally expensive tasks—such as embedding computation or model training—are dispatched to background workers via Redis Queue (rq), which manages four worker processes organized into priority queues (high, default, low). Computed embeddings are cached in a PostgreSQL database to avoid redundant recomputation. File storage is handled by SeaweedFS, a distributed object store. GPU-accelerated inference is provided by NVIDIA Triton Inference Server, which serves pre-trained PLM and prediction models. Prometheus collects metrics for monitoring. In total, the Docker Compose deployment consists of seven or more interconnected services.

The asynchronous task pattern follows a polling model: the client submits a request via HTTP POST, receives a `task_id` in response, and subsequently polls the `/task_status/{task_id}` endpoint until the task reaches a terminal state. This decouples long-running computations from the request-response cycle.

The server exposes nine API modules under `/api/v1/`. The *embeddings* module computes PLM embeddings, caches them in PostgreSQL, and supports HDF5 import and export. The *predictions* module serves nine pre-trained Open Neural Network Exchange (ONNX) models including TMbed, SETH, BindEmbed, and VespaG for tasks such as transmembrane topology prediction and variant effect estimation. The *projections* module performs dimensionality reduction via Principal Component Analy-

sis (PCA), Uniform Manifold Approximation and Projection (UMAP), and t-distributed Stochastic Neighbor Embedding (t-SNE) using the ProtSpace library. The *custom models* module enables users to train and evaluate ML models on their own data via biotrainer. The *active learning* module supports Bayesian optimization with Gaussian Process (GP) and Feedforward Neural Network (FNN) surrogate models for iterative protein engineering. The *PLM eval* module provides automated benchmarking of protein language models. The *PPI* module handles protein-protein interaction dataset analysis, and the *proteins* module provides taxonomy lookup functionality.

The server applies middleware for cross-origin resource sharing, rate limiting via FastAPI-Limiter, request body size limits of 200 MB, and user management through request headers.

2.3 Software Testing Fundamentals

The test pyramid is a widely adopted model for organizing automated tests into layers of increasing scope and decreasing execution speed. At the base, unit tests verify individual functions or classes in isolation, using mocking and stubbing to replace external dependencies. Unit tests execute quickly, typically in milliseconds, and provide fast feedback on localized code changes. The middle layer consists of integration tests, which verify interactions between components: database queries return expected results, API endpoints honor their contracts, and asynchronous task flows complete correctly. At the top, system or end-to-end tests exercise the full application stack.

Continuous integration (CI) is the practice of running automated tests on every code commit, providing rapid feedback to developers and preventing regressions from reaching production. The pytest framework, the standard testing tool for Python, supports this workflow through markers that classify tests (e.g., `@pytest.mark.integration`, `@pytest.mark.slow`), fixtures for reusable setup and teardown logic, parametrization for running the same test logic across multiple inputs, and a hierarchical `conftest.py` system for sharing fixtures across test modules.

Code coverage measures the fraction of source code lines or branches exercised by a test suite. While coverage is a useful proxy for test thoroughness, it does not measure fault-detection capability: a test suite may execute every line without checking that the outputs are correct. Mutation testing addresses this gap by injecting small syntactic faults—called mutants—into the source code (e.g., replacing + with -, negating conditionals, deleting statements) and checking whether any test fails. The mutation score, defined as the ratio of killed mutants to total mutants, provides a more meaningful measure of test suite quality than coverage alone. The mutmut tool implements mutation testing for Python.

2.4 Challenges of Testing ML Systems

Testing ML systems differs fundamentally from testing conventional software due to several characteristics that violate standard testing assumptions.

The *oracle problem* is the most fundamental challenge. For a conventional function, the developer can specify the expected output for a given input and write an assertion to verify it. For protein embeddings, no such ground truth exists: there is no reference embedding vector against which a computed embedding can be verified as correct. It is not possible to write `assert embedding == expected` because the expected value is unknown.

Non-determinism arises from the hardware and software stack used for inference. GPU floating-point arithmetic is non-associative—the order in which additions are performed affects the result at the level of least-significant bits. cuDNN algorithm selection, thread scheduling, and batch-level parallelism introduce further sources of variation. Identical inputs may therefore produce numerically different outputs across runs, even on the same hardware.

Data dependency distinguishes ML systems from logic-driven software. Model behavior is determined not only by code but also by the training data and learned weights. Bugs may arise from data quality issues, distribution shift, or weight corruption, none of which are detectable by traditional unit tests that only exercise code paths.

Computational cost is a practical constraint. Embedding a single protein sequence with a large transformer model takes seconds on a GPU; running a full test suite with real model inference over hundreds of sequences can require hours. This cost makes exhaustive testing impractical within typical CI time budgets.

Model opacity means that the internal state of a transformer model is not directly interpretable. Testing must therefore focus on observable behavioral properties of inputs and outputs rather than on internal representations.

Infrastructure complexity in multi-service deployments introduces failure modes absent in monolithic software. The interaction between the web server, task workers, GPU inference server, database, and file storage creates a large surface area for integration faults.

Finally, *versioning* requirements are stringent: model weights, tokenizers, and inference frameworks such as ONNX Runtime and Triton must all be version-consistent to produce correct results.

2.5 Metamorphic Testing

Metamorphic testing is a testing technique that verifies *relations* between inputs and outputs across multiple executions, rather than checking individual outputs against expected values [CCY98]. A metamorphic relation (MR) specifies that if an input is transformed in a particular way, the output should change—or remain unchanged—in a predictable manner. The key advantage is that metamorphic testing circumvents the oracle problem: it does not require knowing the correct output for any individual input.

A classic example illustrates the principle. The identity $\sin(x) = \sin(\pi - x)$ can be verified without knowing the value of $\sin(x)$ for any specific x: one simply computes both sides and checks equality. The relation between the two computations serves as the test oracle.

In the context of ML systems, metamorphic relations encode expected behavioral invariances. For image classifiers, rotating an image by a small angle should not change the predicted class. For protein language model embeddings, replacing amino acid residues with X (the unknown token) should cause the embedding to diverge monotonically from the original as the replacement fraction increases. Reversing the amino acid order should alter the embedding, since transformer positional encodings are sensitive to sequence direction. Substituting residues with specific amino acids should produce sensitivity patterns that reflect the physicochemical distinctiveness of the substitution.

Metamorphic testing is distinct from property-based testing in that metamorphic relations relate *multiple* executions of the system under test, whereas properties constrain *single* executions. The original metamorphic testing framework was proposed by Chen et al. [CCY98], and subsequent surveys have catalogued applications across diverse domains [Seg+16]. Applications to ML systems have been explored by Xie et al. [Xie+11], who defined relations based on data permutation, addition, and duplication for k-nearest-neighbor and naïve Bayes classifiers.

2.6 Property-Based and Oracle Testing

Property-based testing specifies invariants that must hold for all inputs, rather than enumerating specific input-output pairs. Tools such as Hypothesis for Python and QuickCheck for Haskell generate random inputs and verify that declared properties are satisfied. This approach complements example-based testing by exploring a broader input space and exposing edge cases that manual test design may overlook.

The oracle pattern encapsulates a reusable invariant check in a class with a `check()`

method that returns a structured result containing a pass or fail verdict together with the observed metric value. This pattern is particularly useful for embedding systems, where several invariants can be defined despite the absence of ground-truth outputs. Determinism requires that the same input produces the same output within a specified tolerance. Batch invariance asserts that the embedding of a sequence is unchanged regardless of which other sequences appear in the same batch. Output validity verifies that classification probabilities lie in $[0, 1]$ and sum to one, or that regression outputs are finite. Shape invariance confirms that output dimensions match the model specification. Distance preservation checks that pairwise distances in input space correlate with distances in the projected embedding space.

Because ML outputs are subject to numerical noise from floating-point arithmetic and GPU non-determinism, oracle tests require statistical robustness. Tolerance thresholds define acceptable deviation. Confidence intervals, computed via the t-distribution, quantify uncertainty in mean estimates. Effect sizes such as Cohen's d measure the practical significance of observed differences. Significance tests including the Mann-Whitney U test provide non-parametric comparison of distributions. These statistical utilities enable quantitative assertions on ML outputs that account for inherent variability.

Combining oracle tests with metamorphic relations yields a comprehensive behavioral test suite. Oracle tests verify single-execution invariants (e.g., determinism, valid output ranges), while metamorphic relations verify cross-execution relations (e.g., increasing divergence under progressive masking). Together, they provide meaningful test coverage for systems where traditional assertions are inapplicable.

3 Related Work

This chapter surveys existing work on testing ML systems, metamorphic testing for deep learning, testing practices in scientific and bioinformatics software, and property-based testing for data pipelines. It concludes by identifying the research gap that this thesis addresses.

3.1 Testing Machine Learning Systems

The growing deployment of ML systems in safety-critical and production settings has motivated a substantial body of work on testing techniques tailored to these systems. Tian et al. proposed DeepTest, which applies metamorphic transformations—such as blurring, rain effects, and fog—to input images for autonomous driving Deep Neural Network (DNN)s and checks whether the steering angle prediction changes beyond a threshold [Tia+18]. Pei et al. introduced DeepXplore, which defines neuron coverage as a test adequacy criterion and uses differential testing across multiple DNNs to find inputs that trigger disagreements [Pei+17]. Odena and Goodfellow developed TensorFuzz, which extends coverage-guided fuzzing to neural networks by tracking approximate nearest-neighbor coverage of internal activations [OG19].

Zhang et al. provided a comprehensive survey on ML testing, offering a taxonomy of test adequacy criteria, test input generation methods, and oracle problem solutions [Zha+22]. Braiek and Khomh examined the challenges and emerging practices for testing ML-based systems, including the need for domain-specific test oracles and robustness evaluation [BK20].

A key observation from this literature is that the majority of work focuses on image classification and autonomous driving applications. Comparatively little attention has been paid to embedding models, sequence-based ML systems, or scientific software that integrates PLMs into production pipelines.

3.2 Metamorphic Testing for Deep Learning

Metamorphic testing was originally proposed by Chen et al. as a general-purpose technique for alleviating the oracle problem [CCY98]. Segura et al. later published a

comprehensive survey cataloguing metamorphic testing applications across domains including web services, databases, compilers, and numerical programs [Seg+16].

The application of metamorphic testing to ML was pioneered by Xie et al., who defined relations based on data permutation, addition, and duplication for k-nearest-neighbor and naïve Bayes classifiers [Xie+11]. Murphy et al. proposed additive and multiplicative metamorphic relations for testing ML programs [Mur+08]. Dwarakanath et al. applied metamorphic testing to industrial image classifiers, demonstrating its effectiveness in identifying bugs in production models at a large enterprise [Dwa+18].

These works establish the theoretical and practical foundation for metamorphic testing of ML systems. However, the metamorphic relations proposed in prior work target classifiers operating on images or tabular data. Relations specifically designed for protein language model embeddings—such as progressive masking with the unknown token X, amino acid mutation sensitivity, and sequence reversal—have not been explored and represent a novel contribution of this thesis.

3.3 Testing Scientific and Bioinformatics Software

Scientific software presents distinctive testing challenges. Kanewala and Bieman identified the oracle problem, floating-point precision issues, and the limited overlap between domain expertise and testing expertise as key obstacles [KB14]. Hook and Kelly explored the application of mutation testing to scientific software, demonstrating that syntactic mutations can expose weaknesses in test suites for numerical codes [HK09]. Storer discussed the broader gap between software engineering practices and the norms of scientific computing, arguing that testing culture in scientific projects lags behind industry standards [Sto17].

In the bioinformatics domain specifically, most widely-used tools rely on example-based tests and manual validation against known datasets rather than systematic property-based or metamorphic approaches. Domain-specific tools used by the Biocentral platform, such as biotrainer for model training and bio-embeddings for embedding computation, maintain their own test suites, but these follow conventional testing patterns and do not address the ML-specific challenges outlined in Section 2.4.

3.4 Property-Based Testing for Data Pipelines

Property-based testing, introduced by Claessen and Hughes through the QuickCheck library for Haskell [CH00], specifies invariants over random inputs rather than enumerating specific test cases. MacIver et al. brought this paradigm to Python through

the Hypothesis library, which provides sophisticated input generation and shrinking strategies [MH+19].

Property-based testing has been applied to data pipelines for verifying schema validity, transformation idempotency, and commutativity of data processing steps. However, its application to ML inference pipelines specifically—verifying properties such as batch invariance, output validity bounds, and determinism under floating-point noise—remains limited in the existing literature.

3.5 Research Gap

The review of related work reveals several gaps that this thesis addresses. First, no prior work specifically targets the testing of PLM-based embedding services with a multi-layer test strategy that spans from unit tests through metamorphic experiments. Second, the trade-off between CI execution time and fault-detection capability for property-based testing of ML services has not been quantified. Third, the metamorphic relations developed in this thesis—progressive X-masking divergence, amino acid mutation sensitivity across all 20 standard residues, and sequence reversal sensitivity—are novel contributions not found in the metamorphic testing literature. Fourth, GPU-free deterministic testing infrastructure for embedding services, embodied in the FixedEmbedder design, addresses a practical gap not treated in existing work. This thesis fills these gaps by combining classical software engineering testing with ML-specific techniques, applied to and evaluated on a real-world production system.

4 Approach

4.1 Overall Test Architecture

- Multi-layer test pyramid tailored for embedding-driven ML services
- 5 layers: unit (mocked) → integration (live server) → property/oracle (invariants) → performance (benchmarks) → metamorphic experiments (MRs at scale)
- Total: ∼180 test functions across 24 test files
- Pytest markers for selective execution: `@integration`, `@slow`, `@gpu`, `@performance`, `@property`
- CI integration: markers enable running fast tests on every commit, expensive tests nightly/on-demand
- Figure idea: test pyramid diagram with layer names, test counts, and approximate execution times

4.2 Test Infrastructure

4.2.1 Canonical Test Dataset

- 37 protein sequences in `test_sequences.fasta`, organized into 8 categories
- Categories: standard (3 typical proteins), length variants (1–400 aa), unknown token variants (7 sequences with X, B, Z, J, U, O), all 20 standard AAs, homopolymers, composition extremes, ambiguous codes, structural motifs, real-world proteins (insulin B chain, ubiquitin, GFP core)
- Pydantic-based loader (`test_dataset.py`) with category filtering: `get_test_sequences(categories=''edge_case'')`
- Design rationale: cover normal cases, boundary conditions, and biologically meaningful edge cases in a single reusable dataset

4.2.2 FixedEmbedder

- Problem: real PLM inference (ESM2, ProtT5) requires GPU and takes seconds per sequence — too slow and expensive for CI
- Solution: deterministic, GPU-free embedder that produces embeddings with correct dimensionality and reproducible values
- Algorithm:
  1. Hash the input sequence via SHA-256 to obtain a deterministic seed
  2. Seed a NumPy random number generator with the hash
  3. Generate per-amino-acid base embeddings combined with positional noise
  4. Output shape matches the target PLM: e.g., ProtT5 → 1024 dims, ESM2-T33 → 1280 dims
- Singleton registry (`FixedEmbedderRegistry`) avoids redundant instantiation
- `strict_dataset` mode: optionally validates sequences against the canonical FASTA
- Key property: same sequence → same embedding, regardless of call context, batch, or machine
- 341 lines of implementation; 36 dedicated unit tests verifying determinism, dimensions, edge cases, and embedding quality (no zeros/NaN/Inf)
- Figure idea: diagram showing SHA-256 → seed → RNG → embedding pipeline

4.2.3 Integration Test Infrastructure

- 832-line `conftest.py` providing the integration test framework
- `poll_task()`: async task polling with configurable timeout, terminal status detection, consecutive error tracking
- `FakeResponse` class: intercepts outgoing HTTP calls, generates embeddings via FixedEmbedder, writes directly to PostgreSQL — enables full end-to-end flow testing without GPU
- Monkey-patching: replaces real HTTP embedding calls with FixedEmbedder at the request level

- Session-scoped fixtures for expensive resources: DB connections, HTTP clients, sequence data
- Environment variables: `CI_SERVER_URL` (server address), `CI_EMBEDDER` (mode: `esm2_t6_8m` or `fixed`)

4.3 Unit Tests

- 63 unit tests across 4 test files
- FastAPI `TestClient` with fully mocked backends: `MockedTaskManager`, `UserManager`, `FileManager`, `EmbeddingDatabaseFactory`, Redis-based `RateLimiter`
- Tests validate request/response contracts without starting the real server
- Endpoint coverage:
  - Embeddings (13 tests): embedder listing, embed requests (valid/empty/missing), missing embeddings (valid/invalid JSON/non-dict), HDF5 upload (valid/invalid base64)
  - Predictions (8 tests): model metadata, predict requests (valid/unknown/empty models/empty seqs/too short/too long sequences)
  - Projections (6 tests): config listing, project requests (valid/unknown method/missing fields/custom config passthrough)
- FixedEmbedder unit tests (36 tests): determinism (same seq, different instances, different seeds, pooled, batch→individual consistency), dimension correctness (8 model variants), edge cases (empty/single residue/very long/unknown AA/all AAs/homopolymer/lowercase), embedding quality (no zeros/NaN/Inf, float32, reasonable magnitude, position-sensitivity)
- Design principle: test validation logic and error handling exhaustively; mock all I/O and compute

4.4 Integration Tests

- 56 integration tests across 5 test files
- Run against a live server via Docker Compose (`docker-compose.dev.yml`)
- Use `httpx` with retry logic for resilience against transient failures

- Endpoint flow tests:
  - Embedding flow (7 tests): embedder listing, rejection flows, full embed → poll → cache verification, multi-embedder
  - Prediction flow (8 tests): metadata structure/consistency, rejection flows, full predict task completion
  - Projection flow (5 tests): config retrieval, rejection flows, PCA/UMAP task completion with cache verification
  - Training/inference flow (16 tests): config options (classification/regression), config verify, training start (2 protocols + error cases), inference, model files, **full train → inference flow** with prediction verification, train/val split
- Triton standalone tests (20 tests): all embedding models (ProtT5, ESM2-T33, ESM2-T36) pooled + per-residue; all prediction models (SecondaryStructure, Conservation, BindEmbed, TMbed, SETH, LightAttention, VespaG); model availability checks. Requires Docker with 24 GB+ RAM.
- These tests are the **main contribution**: they validate real end-to-end system behavior including database caching, task queuing, and result retrieval

4.5 Property-Based Oracle Tests

- 21 oracle tests across 3 test files
- Oracle pattern: reusable class with `check()` method returning structured result (pass/fail + metric value)
- Statistical utilities (`statistics.py`): `compute_confidence_interval` (t-distribution), `compute_cohens_d`, `derive_significance_threshold` (k-sigma), `run_with_replicates`, `mann_whitney_test`
- Metric utilities (`embedding_metrics.py`): cosine distance, L2 distance, CSV report generation
- Specific oracles and their tolerances:
  - `BatchInvarianceOracle`: embedding identical regardless of batch composition (cosine $\leq 10^{-6}$)
  - `PredictionDeterminismOracle`: same input → bit-identical output for ONNX models (exact match)

  - `OutputValidityOracle`: probabilities sum to 1, values in $[0, 1]$ (classification) or finite (regression); tolerance 10^{-5}
  - `ShapeInvarianceOracle`: output shape = (seq_len, n_classes) (exact)
  - `ProjectionDeterminismOracle`: PCA bit-identical; UMAP Procrustes distance ≤ 0.3
  - `DimensionalityOracle`: output matches requested dimensions (exact)
  - `ProjectionValueValidityOracle`: no NaN/Inf, finite values
  - `DistancePreservationOracle`: Spearman correlation ≥ 0.3 between input and projected distances

- Models tested directly via ONNX (bypassing server): ProtT5SecondaryStructure (3 classes), BindEmbed (3 classes), TMbed (7 classes), Seth (regression)

4.6 Metamorphic Experiments

- 8 experiments implemented as pytest scripts, producing CSV reports in `tests/reports/`
- All use ESM2-T6-8M as the primary model; one-hot encoding as baseline
- UniRef50-scale validation: 1,750 real protein sequences across 7 length bins (50–1000 aa), fetched via UniRef50 REST API

4.6.1 Idempotency

- Same sequence embedded multiple times must return identical results
- Tolerance: cosine distance $\leq 10^{-5}$ (allows GPU non-determinism)
- 10 sequences $\times$ 4 repeats

4.6.2 Batch Invariance

- Embedding invariant to batch composition: embed(A, B) vs. embed(A) alone
- Transformer self-attention operates within single sequences, not across batch — batch composition must be invisible
- Batch sizes: 2, 5, 10, 20; target sequence at random positions
- Pooled variant: cosine ≤ 0.01; per-residue variant: max absolute difference $\leq 10^{-5}$

4.6.3 Progressive X-Masking

- MR: progressively replace amino acids with X (unknown token) at 14 masking ratios (0–100%)
- Measure cosine distance from original embedding as a function of masking ratio
- Questions: Is divergence monotonically increasing? At what ratio r^* does cosine distance exceed 0.1?
- Progressive (superset) vs. random (independent) masking variants
- 4 variants: basic (4 sequences $\times$ 5 runs), UniRef50 scale (250 seqs/bin $\times$ 7 bins $\times$ 2 runs), one-hot baseline (4 seqs), one-hot UniRef50
- Monotonicity violations allowed $< 30\%$
- Figure idea: cosine similarity vs. masking ratio curve, ESM2 vs. one-hot vs. random baseline

4.6.4 Reversed Sequence

- MR: reverse amino acid order and compare embeddings
- ESM2 expected to be order-sensitive (positional encoding) $\rightarrow$ cosine distance > 0.001
- One-hot baseline expected to be order-insensitive $\rightarrow$ cosine distance $< 10^{-6}$
- Double-reversal identity check: reverse(reverse(seq)) $=$ seq $\rightarrow$ cosine $\leq 10^{-5}$
- Skips palindromic and repetitive sequences

4.6.5 Amino Acid Mutation Sensitivity

- MR: replace residues with each of the 20 standard amino acids at 6 masking ratios (0–100%)
- Questions: Which AA substitutions cause largest embedding change? Does sensitivity correlate with physicochemical properties (charged, hydrophobic, polar)?
- Comparison to X-masking: structured substitution vs. unknown token

- Basic variant: 4 seqs $\times$ 20 AAs $\times$ 3 runs; UniRef50 variant: 250 seqs/bin $\times$ 6 representative AAs (A, D, G, K, L, W)
- Figure ideas: heatmap (AA $\times$ masking ratio), line overlay (all 20 AAs), bar ranking by physicochemical group

4.6.6 Random Baseline

- X-masking on randomly generated (non-biological) sequences: uniform sampling from 20 standard AAs
- 7 length bins (50–1000 aa) $\times$ 50 random sequences $\times$ 2 runs
- Purpose: determine whether X-masking divergence profile differs for "meaningless" vs. real protein sequences

4.6.7 Projection Determinism

- PCA (2D, 3D): expect bit-identical coordinates across runs
- UMAP, t-SNE: stochastic $\rightarrow$ check structural similarity via Procrustes distance (≤ 0.3)

4.6.8 Training Determinism

- Same sequences + same model + same seed $\rightarrow$ identical embeddings across 3 repeated training preparations
- 8 sequences with binary labels; tolerance: cosine $\leq 10^{-5}$

4.7 Performance Tests

- 23 performance tests across 3 test files, using real ESM2-T6-8M model
- Throughput tests (10 tests): latency at 4 sequence lengths (10/15/211/400 aa), canonical dataset throughput, batch throughput, pooled embedding latency, ESM2 vs. FixedEmbedder comparison, scaling report with JSON output
- Memory tests (8 tests): leak detection (repeated single/batch with GC), footprint verification (`nbytes == seq_len × dim × 4`), pooled vs. per-residue memory ($< 10\%$ difference), memory-per-dimension ratio, batch memory estimation

- Scaling tests (5 tests): O(n) linear scaling with sequence length (ratio $< 3\times$), linear scaling with batch size, sequential vs. batch comparison
- Purpose: establish baseline performance characteristics and detect regressions

4.8 Mutation Testing

- Tool: mutmut (Python mutation testing framework)
- Scope: mutates `biocentral_server/` source code; runs only unit tests (excludes slow/integration/performance)
- Mutation operators: arithmetic, boundary, negation, statement deletion, constant replacement
- Mutation score = killed mutants / total mutants
- Purpose: measure how well the unit test suite detects injected faults — complements code coverage
- Configuration in `tests/setup.cfg`

5 Evaluation

5.1 Experimental Setup

- Hardware: specify CPU, GPU (model, VRAM), RAM
- Software: Python version, PyTorch version, ONNX Runtime version, Docker version, Triton Inference Server version
- Primary PLM: ESM2-T6-8M (8 million parameters) — chosen for fast iteration while retaining transformer behavior
- Baseline encodings: one-hot encoding, BLOSUM62
- Docker Compose configuration: server, 4 workers, Redis, PostgreSQL, SeaweedFS, Triton
- All experiments run on the same machine for fair comparison; report wall-clock times

5.2 Test Coverage and Mutation Testing Results

- Total test count: $\sim$180 test functions across 24 files
- Code coverage (line coverage, branch coverage) for unit tests — report numbers with `pytest-cov`
- Mutation testing results:
  - Total mutants generated by mutmut
  - Killed vs. survived mutants
  - Mutation score (%)
  - Which types of mutants survived — what does this reveal about test suite weaknesses?
  - Comparison: mutation score vs. line coverage — are they correlated?

- Table idea: mutation score breakdown by module (embeddings, predictions, projections, etc.)

5.3 CI Execution Time Analysis

- Wall-clock execution time per test category:
  - Unit tests (with mocks): expected ∼seconds
  - Integration tests (with FixedEmbedder): expected ∼minutes
  - Integration tests (with real ESM2): expected ∼longer
  - Property/oracle tests: time per oracle
  - Performance tests: time per benchmark
  - Metamorphic experiments (basic): time for 4-sequence variants
  - Metamorphic experiments (UniRef50 scale): time for 1,750-sequence variants
- **Key analysis**: plot fault-detection value (mutation score contribution, bug-finding capability) against CI time cost
- Trade-off curve: diminishing returns — at what point does adding more property tests stop being worth the CI time?
- FixedEmbedder speedup: how much faster is CI with fixed vs. real embedder? What coverage is sacrificed?
- Recommendation: optimal test portfolio for different CI budgets (e.g., per-commit: unit + fixed integration; nightly: full suite; weekly: UniRef50 experiments)
- Figure idea: stacked bar chart of CI time per layer; scatter plot of time vs. coverage contribution

5.4 Metamorphic Experiment Results

5.4.1 X-Masking Results

- Cosine divergence profiles: plot cosine distance vs. masking ratio for ESM2
- Critical ratio r^*: at what masking percentage does cosine distance exceed 0.1?
- Monotonicity analysis: percentage of sequences with non-monotonic divergence; characterize violations

- ESM2 vs. one-hot comparison: one-hot should show linear divergence (bag-of-features); ESM2 shows non-linear (contextual model)
- ESM2 vs. random baseline: do random sequences diverge differently than real proteins?
- UniRef50 length-bin breakdown: does divergence profile depend on sequence length?
- Figure ideas: cosine similarity curve (ESM2 + one-hot + random), per-bin breakdown, L2 distance dual panel

5.4.2 Reversed Sequence Results

- Distribution of cosine distances for ESM2 reversed sequences — confirm order-sensitivity
- One-hot baseline: confirm near-zero distance (order-insensitive, as expected)
- Double-reversal identity: confirm recovery within tolerance
- Interpretation: positional encoding in ESM2 captures sequence directionality

5.4.3 AA Mutation Sensitivity Results

- Heatmap: amino acid $\times$ masking ratio showing cosine distance
- Sensitivity ranking: which AAs cause largest/smallest embedding changes at $\sim$10% mutation?
- Physicochemical grouping: charged (D, E, K, R) vs. hydrophobic (A, V, L, I) vs. polar (S, T, N, Q) vs. special (G, P, C, W)
- Comparison to X-masking: is replacing with a real AA more or less disruptive than replacing with X?
- UniRef50 scale: do sensitivity patterns hold across diverse real proteins?
- Figure ideas: heatmap, bar chart by physicochemical group, mutation vs. X-masking overlay

5.5 Oracle Test Findings

- Batch invariance: report cosine distances (expect $\ll 10^{-6}$); any violations?
- Prediction determinism: confirm bit-identical ONNX outputs across runs
- Output validity: any probabilities outside $[0, 1]$ or not summing to 1? Any NaN/Inf?
- Shape invariance: any dimension mismatches?
- Projection distance preservation: Spearman correlation values for PCA, UMAP, t-SNE
- Projection determinism: PCA exact; UMAP/t-SNE Procrustes distances
- Table idea: oracle name, metric, threshold, observed value, pass/fail

5.6 Performance Results

- Latency by sequence length: plot latency vs. sequence length (10, 15, 211, 400 aa)
- Throughput: sequences per second for batch processing
- Memory: footprint per embedding, leak detection results
- $O(n)$ scaling: confirm linear scaling with sequence length (ratio $< 3\times$)
- Batch efficiency: batch vs. sequential processing speedup
- ESM2 vs. FixedEmbedder: orders-of-magnitude speed difference quantified
- Table/figure idea: latency table, memory bar chart, scaling line plot

5.7 Discussion of Results

- **Answering the RQ**: What is the trade-off between CI execution time and test coverage?
  - Unit tests: fastest, catch validation/contract bugs, limited for ML-specific issues
  - Integration tests (FixedEmbedder): moderate time, catch real flow bugs without GPU — best bang-for-buck

  - Oracle/property tests: moderate time, catch invariant violations that unit tests miss
  - Metamorphic experiments: slow (especially UniRef50), provide unique behavioral insights but diminishing returns for CI
  - Performance tests: moderate time, essential for regression detection

- Recommended CI strategy: per-commit (unit + fixed integration), nightly (+ oracles + performance), weekly/release (+ metamorphic + real GPU)
- FixedEmbedder as enabler: makes the pyramid viable by decoupling CI from GPU availability
- Surprising findings (if any): unexpected oracle failures, non-monotonic masking behavior, etc.

6 Discussion

6.1 Trade-Off Summary

- Restate the RQ and summarize the answer with concrete numbers from Chapter 5
- Optimal allocation of CI budget across test layers
- The FixedEmbedder as a key enabler: percentage of integration tests that can run GPU-free; what fraction of real bugs does it catch vs. miss?
- Diminishing returns: at what point does adding more metamorphic tests stop improving fault detection?
- Cost of false security: what types of bugs can only be caught with real GPU inference?

6.2 Generalizability

- Beyond biocentral: applicability to other PLM-based services (e.g., ColabFold, bio-embeddings servers)
- Beyond protein embeddings: applicability to NLP embedding services (sentence transformers, BERT-based APIs)
- Beyond embeddings: applicability to other ML inference services (image classifiers, recommendation systems)
- Which parts of the approach are domain-specific (canonical protein dataset, AA mutation MRs) vs. generalizable (test pyramid, FixedEmbedder pattern, oracle framework)?
- Multi-service testing: the approach handles complex Docker deployments (server + workers + GPU + DB) — relevant for any microservice-based ML system

6.3 Limitations

- Single PLM for experiments: ESM2-T6-8M is the smallest ESM2 variant; larger models (650M, 3B) or different architectures (ProtT5) may exhibit different behavior
- No adversarial testing: the metamorphic experiments use structured transformations, not adversarial inputs designed to fool models
- FixedEmbedder expressiveness: SHA-256-based embeddings do not capture real biological properties — bugs that require biologically meaningful embeddings will be missed
- No end-user study: the test suite was developed by the author; no external developer usability evaluation
- Limited mutation testing scope: mutmut only targets unit tests; integration/property tests not evaluated for mutation score
- Hardware-specific timings: execution times depend on specific CPU/GPU; relative ratios are more meaningful than absolutes

6.4 Threats to Validity

- Internal validity:
  - GPU non-determinism tolerance thresholds were chosen empirically — different thresholds could change pass/fail outcomes
  - FixedEmbedder monkey-patching may mask real HTTP/serialization bugs
  - Mutation testing results depend on mutmut's mutation operators — different tools may produce different scores
- External validity:
  - Results specific to biocentral's architecture (FastAPI + Redis + Triton) — other frameworks may have different failure modes
  - ESM2-T6-8M behavior may not generalize to larger or differently trained models
  - UniRef50 sequences are biased toward well-studied proteins — rare/novel proteins underrepresented
- Construct validity:

- "Test coverage" measured via mutation score and code coverage — other metrics (fault detection rate in production) would be more meaningful but are unavailable
- "CI execution time" measured on a single machine — cloud CI environments may differ

7 Conclusion

7.1 Summary of Contributions

- Multi-layer test pyramid for embedding-driven ML services: unit → integration → property/oracle → performance → metamorphic experiments
- FixedEmbedder: deterministic, GPU-free test infrastructure enabling fast CI for PLM-based services
- 8 metamorphic experiments for PLM embeddings: X-masking divergence, reversed sequence sensitivity, AA mutation sensitivity, with validation at UniRef50 scale (1,750 real proteins)
- Oracle test pattern with statistical utilities for robust invariant checking under numerical noise
- Mutation testing evaluation quantifying unit test fault-detection capability
- Quantitative analysis of the time-vs-coverage trade-off: concrete recommendations for CI test portfolios

7.2 Key Findings

- Summarize the most important result for the RQ (e.g., "Integration tests with FixedEmbedder provide X% of fault detection at Y% of the CI time of full GPU tests")
- Metamorphic experiments reveal: [insert key behavioral findings about ESM2, e.g., critical masking ratio, sensitivity ranking]
- Oracle tests confirm: [insert which invariants hold/fail, e.g., batch invariance holds perfectly, projection distance preservation varies by method]
- Mutation testing reveals: [insert mutation score and what it implies about test adequacy]

7.3 Future Work

- Extend metamorphic experiments to larger PLMs: ESM2-650M, ESM2-3B, ProtT5-XL — do behavioral properties change with model scale?
- Adversarial testing: design inputs that maximize embedding instability or prediction errors
- Automated MR discovery: use search-based techniques to find new metamorphic relations that maximize fault detection
- Coverage-guided property testing: combine mutation testing feedback with property test generation to target untested code
- Cross-platform evaluation: apply the test strategy to other ML serving frameworks (TensorFlow Serving, TorchServe, BentoML)
- Production monitoring: extend oracle tests into runtime monitors that flag anomalous embeddings in production
- Developer study: evaluate the test suite's usability with external developers contributing to biocentral

Abbreviations

PLM Protein Language Model

CI Continuous Integration

API Application Programming Interface

GPU Graphics Processing Unit

ONNX Open Neural Network Exchange

PCA Principal Component Analysis

UMAP Uniform Manifold Approximation and Projection

t-SNE t-distributed Stochastic Neighbor Embedding

GP Gaussian Process

FNN Feedforward Neural Network

RQ Research Question

ML Machine Learning

DNN Deep Neural Network

REST Representational State Transfer

List of Figures

List of Tables

Bibliography

[Ber+25] J. H. Bernardo, D. A. da Costa, F. R. Cogo, S. Q. de Medeiros, and U. Kulesza. *Continuous Integration Practices in Machine Learning Projects: The Practitioners' Perspective*. 2025. arXiv: 2502.17378 [cs.SE].

[BK20] H. Ben Braiek and F. Khomh. "On Testing Machine Learning Programs." In: *Journal of Systems and Software* 164 (2020), p. 110542. DOI: 10.1016/j.jss.2020.110542.

[CCY98] T. Y. Chen, S. C. Cheung, and S. M. Yiu. "Metamorphic Testing: A New Approach for Generating Next Test Cases." In: *Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology*. 1998.

[CH00] K. Claessen and J. Hughes. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs." In: *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 2000, pp. 268–279. DOI: 10.1145/351240.351266.

[Cha+23] J. Chandrasekaran, T. Cody, N. McCarthy, E. Lanus, and L. Freeman. *Test & Evaluation Best Practices for Machine Learning-Enabled Systems*. 2023. arXiv: 2310.06800 [cs.SE].

[Dwa+18] A. Dwarakanath, M. Ahuja, S. Sikand, R. M. Rao, R. P. J. C. Bose, N. Dubey, and S. Podder. "Identifying Implementation Bugs in Machine Learning Based Image Classifiers Using Metamorphic Testing." In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2018, pp. 118–128. DOI: 10.1145/3213846.3213858.

[Eln+21] A. Elnaggar, M. Heinzinger, C. Dallago, G. Rihawi, Y. Wang, L. Jones, T. Gibbs, T. Feher, C. Angerer, M. Steinegger, D. Bhowmik, and B. Rost. *ProtTrans: Towards Cracking the Language of Life's Code Through Self-Supervised Deep Learning and High Performance Computing*. 2021. arXiv: 2007.06225 [cs.LG].

[Fra+25] S. Franz, T. Olenyi, P. Schloetermann, A. Smaoui, L. F. Jimenez-Soto, and B. Rost. *biocentral: embedding-based protein predictions*. Nov. 2025. DOI: 10.1101/2025.11.21.689449.

[HK09] D. Hook and D. Kelly. "Testing for Trustworthiness in Scientific Software." In: *Proceedings of the ICSE Workshop on Software Engineering for Computational Science and Engineering (SECSE)*. 2009, pp. 59–64. DOI: 10.1109/SECSE.2009.5069163.

[KB14] U. Kanewala and J. M. Bieman. "Testing Scientific Software: A Systematic Literature Review." In: *Information and Software Technology* 56.10 (2014), pp. 1219–1232. DOI: 10.1016/j.infsof.2014.05.006.

[Lin+23] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, R. Verkuil, O. Kabeli, Y. Shmueli, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candber, and A. Rives. "Evolutionary-Scale Prediction of Atomic-Level Protein Structure with a Language Model." In: *Science* 379.6637 (2023), pp. 1123–1130. DOI: 10.1126/science.ade2574.

[MH+19] D. R. MacIver, Z. Hatfield-Dodds, et al. "Hypothesis: A New Approach to Property-Based Testing." In: *Journal of Open Source Software* 4.43 (2019), p. 1891. DOI: 10.21105/joss.01891.

[Mur+08] C. Murphy, G. E. Kaiser, L. Hu, and L. Wu. "Properties of Machine Learning Applications for Use in Metamorphic Testing." In: *Proceedings of the 20th International Conference on Software Engineering and Knowledge Engineering (SEKE)*. 2008, pp. 867–872.

[OG19] A. Odena and I. Goodfellow. "TensorFuzz: Debugging Neural Networks with Coverage-Guided Fuzzing." In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 4901–4911.

[Pei+17] K. Pei, Y. Cao, J. Yang, and S. Jana. "DeepXplore: Automated Whitebox Testing of Deep Learning Systems." In: *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. 2017, pp. 1–18. DOI: 10.1145/3132747.3132785.

[Rzi+25] D. E. Rzig, A. Houerbi, R. G. Chavan, and F. Hassan. *Empirical Analysis on CI/CD Pipeline Evolution in Machine Learning Projects*. 2025. arXiv: 2403.12199 [cs.SE].

[Seg+16] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés. "A Survey on Metamorphic Testing." In: *IEEE Transactions on Software Engineering* 42.9 (2016), pp. 805–824. DOI: 10.1109/TSE.2016.2532875.

[Sto17] T. Storer. "Bridging the Chasm: A Survey of Software Engineering Practice in Scientific Programming." In: *ACM Computing Surveys* 50.4 (2017), pp. 1–32. DOI: 10.1145/3084225.

[Suz+15] B. E. Suzek, Y. Wang, H. Huang, P. B. McGarvey, C. H. Wu, and the UniProt Consortium. "UniRef Clusters: A Comprehensive and Scalable Alternative for Improving Sequence Similarity Searches." In: *Bioinformatics* 31.6 (2015), pp. 926–932. DOI: 10.1093/bioinformatics/btu739.

[Tia+18] Y. Tian, K. Pei, S. Jana, and B. Ray. "DeepTest: Automated Testing of DNN-Driven Autonomous Cars." In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 2018, pp. 303–314. DOI: 10.1145/3180155.3180220.

[Xie+11] X. Xie, J. W. K. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen. "Testing and Validating Machine Learning Classifiers by Metamorphic Testing." In: *Journal of Systems and Software* 84.4 (2011), pp. 544–558. DOI: 10.1016/j.jss.2010.11.920.

[Zha+22] J. M. Zhang, M. Harman, L. Ma, and Y. Liu. "Machine Learning Testing: Survey, Landscapes and Horizons." In: *IEEE Transactions on Software Engineering* 48.1 (2022), pp. 1–36. DOI: 10.1109/TSE.2019.2962027.